

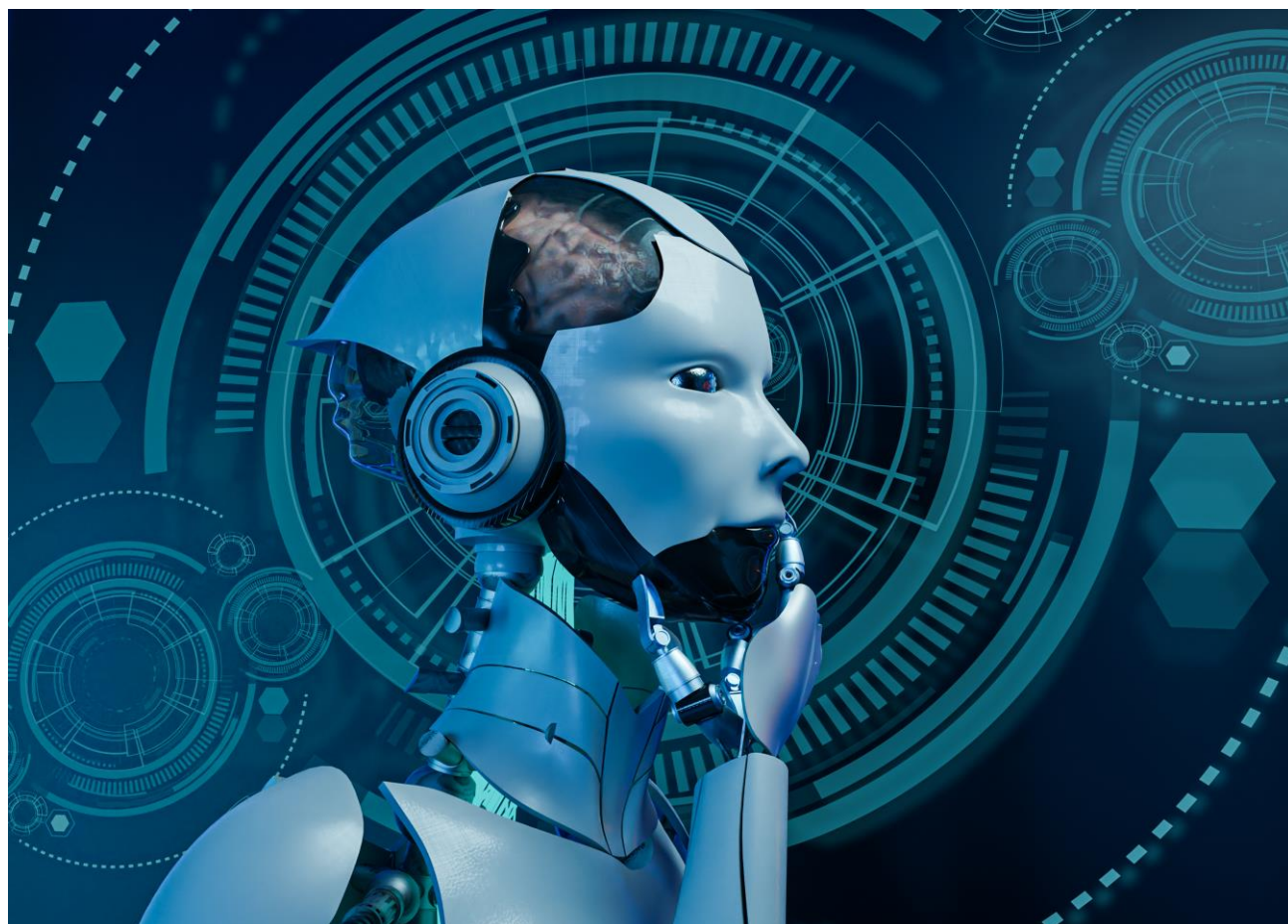
Uma análise empírica e matemática de algoritmos de ordenação baseados em comparação entre as versões originais

# Comparação de Eficiência entre algoritmos de ordenação Clássicas e suas variações.

Raphael Sousa Rabelo Rates

25/08/2024

---



# Resumo

Este trabalho apresenta uma pesquisa de teste entre dois algoritmos de ordenações clássicos, *bubbleSort* e *InsertionSort* e as suas variações modificadas pela ferramenta da OpenAI **ChatGPT**. Escolhendo uma variação gerada pelo ChatGPT, testamos as métricas da função gerada com a função original e comparamos os resultados para medir o quão eficiente é a implementação modificada em comparação ao algoritmo clássico.

## 1 Introdução

Após a aula de algoritmos de ordenação na matéria de **Laboratório de Algoritmo e Estrutura de Dados**, A professora da matéria, Paola Rodrigues de Godoy Accioly, apresentou aos seus alunos um projeto para testar tais algoritmos de ordenação apresentados e selecionar alguns deles para perguntar a IA ChatGPT para tornar tal código mais eficiente. Após a seleção do resultado gerado, comparamos as métricas das respectivas funções para ver o desempenho de do algoritmo gerado em comparação ao material original

Os seguintes algoritmos clássicos apresentados para teste foram totais de 5, sendo possível a escolha de apenas 2 deles para o teste.

|               |           |                                                                            |
|---------------|-----------|----------------------------------------------------------------------------|
| BubbleSort    | Iterativa | Move os maiores elementos para o final do vetor em múltiplas passagens     |
| InsertionSort | Iterativa | Insere os elementos e suas devidas posições por uma lista já ordenada      |
| SelectionSort | Iterativa | Seleciona repetidamente elemento e o coloca na posição correta da lista    |
| MergeSort     | Recursiva | Divide a lista em duas e ordena cada divisão para depois combinar em ordem |
| QuickSort     | Recursiva | Separa a lista em partes menores baseadas em um pivô e ordena por recursão |

---

## 2 Fundamentação Teórica

### 2.1 Treinamento

Durante essa etapa, é selecionado os dois algoritmos de ordenação para perguntar ao LLM da OpenAI sobre como apresentar uma possível melhoria de eficiência no código original. Após a escolha dos algoritmos modificados, é realizada uma análise sobre a escolha do código modificado e uma justificação da escolha. Os algoritmos selecionados foram o *BubbleSort* e o *InsertionSort*.

O *BubbleSort* é um algoritmo que ordena pela troca de valores contínua no vetor até que o elemento que é deslocado seja menor que o elemento próximo a posição dele, sendo

```
void bubblesort(int *vet, int n){
    int i, mudou, fim = n-1;
    do{
        mudou = 0;
        for(i = 0; i < fim; i++){
            if(vet[i] > vet[i+1]){
                swap(&vet[i], &vet[i+1]);
                mudou = 1;
            }
        }
        fim--;
    }while(mudou);
}
```

uma ordenação crescente. Sua complexidade no pior dos casos é  $O(n^2)$ , pois após ordenar um vetor de  $n$  elementos, na pior das hipóteses ele terá que deslocar cada elemento, ou seja comparar um elemento  $L[n]$  com  $L[n+1]$ ,  $n$  vezes. No melhor dos casos ele compara os valores do vetor, porém sem necessitar de uma troca de valores. Por mais que o código pareça ter uma lógica simples, sua eficiente em comparação com os outros algoritmos clássicos é bastante inferior.

Outro algoritmo usado para esta pesquisa é o *InsertionSort*, um algoritmo de ordenação que constrói uma matriz final com um elemento com uma inserção por vez, percorrendo os elementos do vetor até achar o valor exato para um elemento  $n$  está em uma certa

posição do vetor. Sua complexidade, assim como o *BubbleSort*, é de  $O(n^2)$  por precisar percorrer todos os elementos do vetor  $n$  vezes para comparação de um elemento.

```
void insertionSort(int *vet, int n){
    int i, j, aux;
    for(i = 1; i < n; i++){
        aux = vet[i];
        j = i;
        while(j > 0 && aux < vet[j-1]){
            vet[j] = vet[j-1];
            j--;
        }
        vet[j] = aux;
    }
}
```

O *insertionSort* não é um algoritmo eficiente em comparação com outros tipos de ordenação como o Merge ou Quick, porém é bastante útil para pequenas entradas.

Cada algoritmo sofreu alterações da ferramenta de chat, sendo escolhida as seguintes variações:

### BUBBLESORT MODIFICADO

```
void bubblesortM(int *vet, int n, unsigned long *comparacoes, unsigned long *trocas) {
    int i, mudou;
    *trocas = 0;
    *comparacoes = 0;
    int sorted = 0;
    while (!sorted) {
        sorted = 1;
        for (i = 0; i <= n - 2; i += 2) {
            (*comparacoes)++;
            if (vet[i] > vet[i + 1]) {
                swap(&vet[i], &vet[i + 1]);
                sorted = 0;
                (*trocas)++;
            }
        }
        for (i = 1; i <= n - 2; i += 2) {
            (*comparacoes)++;
            if (vet[i] > vet[i + 1]) {
                swap(&vet[i], &vet[i + 1]);
                sorted = 0;
                (*trocas)++;
            }
        }
    }
}
```

O algoritmo acima foi escolhido para comparação por dois motivos:

- **Ausência de variações:** Dentre todos os resultados dados pelo ChatGPT, quase todos apenas reescreviam o mesmo algoritmo com apenas algumas modificações não muito relevantes quando testadas em análises empíricas.
- **Lógica de Percorrer o array:** a lógica presente no algoritmo demonstra ser uma comparação entre pares e ímpares, necessitando de menos comparações em comparações com o algoritmo original. Pesquisando mais a fundo, descobre-se que o algoritmo gerado pelo ChatGPT é um algoritmo de ordenação baseado no bubbleSort chamada “*Odd-Even Sort*”, no qual consiste em ser uma versão do algoritmo porém com os pares e ímpares dois grupos no array sendo comparados com os seus respectivos elementos em cada grupo.
- **Análise Matemática:** sabendo que o bubbleSort tem a sua complexidade  $O(n^2)$  e fazendo uma análise do algoritmo no pior caso, temos:

$C_1$  = mudança de valor

$C_2$  = condição

$C_3$  = swap ()

$C_4$  = iniciando ou declarando uma variável

$n$  = quantidade de elementos no array

$$\begin{aligned}
 Eq &= C_1 + n * 2 \left( C_4 + C_2 + n * \frac{(C_2 + C_3 + 2 * C_1)}{2} \right) \\
 &= C_1 + n * (2C_4 + 2C_2 + nC_2 + nC_3 + n2C_1) \\
 &= n^2 2C_2 + n^2 2C_3 + n^2 2C_1 + nC_4 + nC_2 + C_1 \\
 &= n^2 * \left( 2C_2 + 2C_3 + 2C_1 + \frac{C_4}{n} + \frac{C_2}{n} \right) + C_1
 \end{aligned}$$

Sendo  $C_1$ ,  $C_2$ ,  $C_3$  e  $C_4$  constantes, que são desconsiderados em BigO Notation, e que o maior fator de mudança da equação é  $n^2$ , considerando  $\beta = 2C_2 + 2C_3 + 2C_1 + \frac{C_4}{n} + \frac{C_2}{n}$  e uma constante aleatório como 4, podemos dizer que:

$$Eq = n^2 * \beta + C_1 \Rightarrow n^2 * \beta + C_1 \leq 4 * n^2 \quad \text{logo,}$$

Como a equação  $Eq$  é menor ou igual a uma constante multiplicado por  $n^2$ , então a complexidade do pior caso do algoritmo modificado é  $O(n^2)$ , que é a mesma do bubbleSort original.

OBS: na equação é desconsiderada os ponteiros de comparações e trocas para representar um cenário com a ordenação pura e sem contagem de métricas.

---

## INSERTIONSORT MODIFICADO

```
void insertionSortM(int *vet, int n, unsigned long *comparacoes, unsigned long *trocas) {
    int i, j, aux, pos;
    *trocas = 0;
    *comparacoes = 0;
    for (i = 1; i < n; i++) {
        aux = vet[i];
        int low = 0, high = i;
        while (low < high) {
            int mid = (low + high) / 2;
            (*comparacoes)++;
            if (vet[mid] <= aux) {
                low = mid + 1;
            } else {
                high = mid;
            }
        }
        pos = low;
        for (j = i; j > pos; j--) {
            vet[j] = vet[j - 1];
            (*trocas)++;
        }
        vet[pos] = aux;
        if (pos != i) {
            (*trocas)++;
        }
    }
}
```

O algoritmo acima foi escolhido para comparação por dois motivos:

- **Ausência de variações:** Dentre todos os resultados dados pelo ChatGPT, quase todos apenas reescreviam o mesmo algoritmo com apenas algumas modificações não muito relevantes quando testadas em análises empíricas.
- **Busca Binária:** A modificação de atribuir uma busca binária ao invés de percorrer o array por inteiro, otimizando o tempo e o número de comparações entre elementos.
- **Análise Matemática:** sabendo que o insertionSort tem a sua complexidade  $O(n^2)$  e fazendo uma análise do algoritmo modificado no pior caso, temos:

$C_1$  = mudança de valor

$C_2$  = condição

$C_3$  = swap ()

$C_4$  = iniciando ou declarando uma variável

$n$  = quantidade de elementos no array

$$Eq = 4C_1 + n \left( 3C_4 + C_2 + n * \left( \frac{C_2 + C_4}{n^2} \right) + C_4 + n(C_4) + C_4 \right)$$

$$= 4C_1 + n * (5C_4 + C_2 + \log(n) + nC_4)$$

$$= n^2 C_4 + n \log(n) + nC_2 + n5C_4 + 4C_1$$

$$= n^2 * \left( C_4 + \frac{\log(n)}{n} + \frac{C_2}{n} + \frac{5C_4}{n} + \frac{4C_1}{n^2} \right)$$

Sendo  $C_1$ ,  $C_2$ ,  $C_3$  e  $C_4$  constantes, que são desconsiderados em BigO Notation, e que o maior fator de mudança da equação é  $n^2$ , considerando  $\beta = \left( C_4 + \frac{\log(n)}{n} + \frac{C_2}{n} + \frac{5C_4}{n} + \frac{4C_1}{n^2} \right)$  e uma constante aleatório como 3, podemos dizer que:

$$Eq = n^2 * \beta + C_1 \Rightarrow n^2 * \beta + C_1 \leq 3 * n^2 \quad \text{logo,}$$

Como a equação  $Eq$  é menor ou igual a uma constante multiplicado por  $n^2$ , então a complexidade do pior caso do algoritmo modificado é  $O(n^2)$ , que é a mesma do bubbleSort original, mesmo levando em conta que a complexidade das comparações é de  $O(\log(n))$  em comparação ao algoritmo original que é  $O(n^2)$ .

OBS: na equação é desconsiderada os ponteiros de comparações e trocas para representar um cenário com a ordenação pura e sem contagem de métricas.

---

## 3 OBJETIVO GERAL

A finalidade desse projeto é analisar os resultados gerados pela IA para compreender o quão eficiente são os resultados modificados em relação aos algoritmos clássicos, medindo a eficiência do ChatGPT e, melhoria de código já existente.

O treinamento do ChatGPT foi executado com a implementação dos algoritmos originais como parte do prompt para o Chat responder com a entrega de um algoritmo. O algoritmo tem como possibilidade ser mais eficiente ou não.

---

## 4 METODOLOGIA

O método usado para testar os algoritmos será um conjunto de algoritmos em arquivos da linguagem C (arq.c), acompanhados com arquivos de bibliotecas personalizadas (arq.h). Os testes seguirão seguinte estrutura de arquivos.

|              |                                                                                       |
|--------------|---------------------------------------------------------------------------------------|
| biblioteca.c | Arquivo do código fonte das funções de teste para a finalidade do projeto             |
| biblioteca.h | Arquivo de biblioteca do arquivo biblioteca.c                                         |
| main.c       | Arquivo principal onde as funções de teste serão executadas                           |
| ordination.c | Arquivo do código fontes das funções de ordenação                                     |
| ordination.h | Arquivo de biblioteca do ordination.c                                                 |
| style.c      | Arquivo do código fonte de estilização para melhorar a exibição do teste via terminal |
| style.h      | Arquivo de biblioteca do arquivo style.c                                              |

---

### Explicação do Código

#### 4.1 Biblioteca.c

Esse arquivo possui funções que com papeis de alocação, preenchimento, e impressão de vetores, bem como a execução de testes de desempenho em algoritmos de ordenação. Abaixo, apresento uma explicação detalhada das principais funções e seus propósitos.



#### 4.1.1. Função void alocar\_vetor (int \*\*vetor, int n)

```
void alocar_vetor(int **vetor, int n) {
    if (n > 0) {
        *vetor = (int *)malloc(n * sizeof(int));
        if (*vetor == NULL) {
            fprintf(stderr, RED "ERROR:" CYAN " Falha na alocação de memoria." RESET "\n");
            exit(EXIT_FAILURE);
        }
    } else {
        fprintf(stderr, RED "ERROR:" CYAN " Numero de alocação invalido." RESET "\n");
        exit(EXIT_FAILURE);
    }
}
```

- Descrição: Esta função é responsável por alocar dinamicamente a memória para um vetor de inteiros de tamanho n. Caso a alocação falhe ou n seja menor ou igual a zero, a função imprime uma mensagem de erro estilizada e encerra o programa.
    - Entrada: int \*\*vetor: Ponteiro para o vetor a ser alocado.
    - int n: Tamanho do vetor a ser alocado.
  - Saída: Nenhuma (a função aloca memória para o vetor ou encerra o programa em caso de erro).
- 

#### 4.1.2 Função void preencher\_vetor (int \*vetor, int n, int min, int max, int seed offset)

```
void preencher_vetor(int *vetor, int n, int min, int max, int seed_offset) {
    srand(time(0) + seed_offset);
    for (int i = 0; i < n; i++) {
        vetor[i] = min + rand() % (max - min + 1);
    }
}
```

- Descrição: Esta função preenche um vetor de inteiros com valores aleatórios dentro de um intervalo específico definido por min e máx. A semente para a geração de números aleatórios é baseada no tempo atual e em um deslocamento (Seed offset), garantindo diversidade nos valores gerados.
- Entrada:
  - int \*vetor: Ponteiro para o vetor a ser preenchido.

- int n: Tamanho do vetor.
  - int min: Valor mínimo do intervalo de números aleatórios.
  - int máx.: Valor máximo do intervalo de números aleatórios.
  - int Seed offset: Deslocamento aplicado na semente de aleatoriedade.
- Saída: Nenhuma (a função modifica o vetor diretamente).
- 

#### 4.1.3. Função void imprimir\_vetor (const int \*vetor, int n)

```
void imprimir_vetor(const int *vetor, int n) {  
    for (int i = 0; i < n; i++) {  
        if ((i % 10) == 0) { printf("\n"); }  
        printf(GREEN NEGRITO "%d " RESET, vetor[i]);  
    }  
}
```

- Descrição: Esta função imprime os elementos de um vetor de inteiros, formatando a saída de maneira estilizada e organizando os números em linhas de até 10 elementos.
  - Entrada:
    - const int \*vetor: Ponteiro para o vetor a ser impresso.
    - int n: Tamanho do vetor.
  - Saída: Nenhuma (a função imprime os elementos do vetor no console).
- 

#### 4.1.4. Função void testar\_algoritmo(Argslteratctive\* args, int\* elements)

```

void testar_algoritmo(Argslteractive* args, int* elements) {
    clock_t inicio, fim;
    unsigned long comparacoes = 0, trocas = 0;

    inicio = clock();
    args->func(elements, args->n, &comparacoes, &trocas);
    fim = clock();
    unsigned long duracao = (fim - inicio) * 1000 / CLOCKS_PER_SEC;

    args->comparacoes = comparacoes;
    args->trocas = trocas;
    args->duracao = duracao;

    printf("\n NEGRITO RED \"EXECUTANDO\" RESET \" | vetor %d | \" CYAN \"%lu\" RESET \"ns | \" RED \"%d\" RESET \" - \" GREEN \"%d\" RESET \" = \" MAGENTA \"%d\" RESET \" | \" CYAN \"%lu\" RESET \"comparacoes | \" CYAN \"%lu\" RESET \"trocas.\n\",
        args->vetor, args->duracao, args->max, args->min, args->max - args->min, args->comparacoes, args->trocas);
    printf(YELLOW NEGRITO \"=====\" RESET);
}

```

- Descrição: O código tem como objetivo executar e avaliar o desempenho de um algoritmo de ordenação, medindo o tempo de execução, o número de comparações e o número de trocas realizadas. A função recebe como parâmetros uma estrutura Argslteractive, que contém informações necessárias para a execução do algoritmo, e um ponteiro para um vetor de inteiros elements. Inicialmente, são declaradas variáveis para armazenar o tempo de início e fim da execução do algoritmo, bem como as contagens de comparações e trocas, que são inicializadas com zero. Em seguida, a função obtém o tempo de início usando a função clock() e, logo após, chama o algoritmo de ordenação, passando o vetor, o tamanho do vetor e os ponteiros para as variáveis de contagem de comparações e trocas, permitindo que essas informações sejam atualizadas durante a execução.

Após a execução do algoritmo, o tempo de fim é capturado, e a duração da execução é calculada em milissegundos, considerando a diferença entre o tempo de fim e início. As contagens de comparações, trocas e a duração da execução são armazenadas na estrutura args para posterior análise. A função então imprime essas informações no console de maneira formatada, destacando a identificação do vetor, a duração da execução, a diferença entre o maior e o menor valor do vetor, e o número de comparações e trocas realizadas durante a ordenação. Além disso, um separador visual é impresso para organizar a apresentação dos resultados, utilizando códigos de cores ANSI para estilizar a saída, destacando aspectos como o tempo de execução e as operações realizadas.

- Entrada:
  - Argslteractive\* args: ponteiro de uma estrutura que inclui o ponteiro para a função de ordenação, o tamanho do vetor, e variáveis para armazenar os resultados do teste. A estrutura usada se encontra em Biblioteca.h.
  - Int\* elements: ponteiro de um vetor para ordenar os elementos do vetor.

- Saída: Retorna nada (apenas testa o algoritmo e salva os dados do teste)

#### 4.1.5. Função void executar\_teste (int tamanho, FuncaoIterativa func1, FuncaoIterativa func2, const char\* nome\_func1, const char\* nome\_func2)

```
void executar_teste(int tamanho, FuncaoIterativa func1, FuncaoIterativa func2, const char* nome_func1, const char* nome_func2) {
    ArgsIterativo args, args2;
    args.func = func1;
    args2.func = func2;
    args.n = tamanho;
    args2.n = tamanho;
    args.min = ULONG_MAX; args.max = 0;
    args2.min = ULONG_MAX; args2.max = 0;

    unsigned long total_duracao_original = 0, total_duracao_modificado = 0;
    unsigned long duracao_min_original = ULONG_MAX, duracao_max_original = 0;
    unsigned long total_comparacoes_original = 0, total_comparacoes_modificado = 0;
    unsigned long duracao_min_modificado = ULONG_MAX, duracao_max_modificado = 0;
    unsigned long total_trocas_original = 0, total_trocas_modificado = 0;

    for (int i = 0; i < 10; i++) {
        int *elements = NULL, *elements_copia = NULL;
        alocar_vetor(&elements, tamanho);
        alocar_vetor(&elements_copia, tamanho);

        preencher_vetor(elements, tamanho, 0, tamanho * 10, i);
        memcpy(elements_copia, elements, tamanho * sizeof(int));
        args.vetor = i + 1;
        args2.vetor = i + 1;

        printf(CYAN NEGrito "\n\n-----\n" RESET);
        printf(CYAN NEGrito "TESTANDO FUNCAO ORIGINAL %s" RESET);
        printf(CYAN NEGrito "-----\n" RESET);

        unsigned long comparacoes_total_original = 0, trocas_total_original = 0;
        unsigned long soma_duracao_original = 0;

        for (int i = 0; i < 10; i++) {
            testar_algoritmo(&args, elements_copia);
            soma_duracao_original += args.duracao;
            comparacoes_total_original += args.comparacoes;
            trocas_total_original += args.trocas;

            if (args.duracao < duracao_min_original) {
                duracao_min_original = args.duracao;
                args.min = duracao_min_original;
            }
            if (args.duracao > duracao_max_original) {
                duracao_max_original = args.duracao;
                args.max = duracao_max_original;
            }

            memcpy(elements_copia, elements, tamanho * sizeof(int));
        }

        total_duracao_original += soma_duracao_original / 10;
        total_comparacoes_original += comparacoes_total_original / 10;
        total_trocas_original += trocas_total_original / 10;

        printf(CYAN NEGrito "\n\n-----\n" RESET);
        printf(CYAN NEGrito "TESTANDO FUNCAO %s MODIFICADA %s" RESET);
        printf(CYAN NEGrito "-----\n" RESET);

        unsigned long comparacoes_total_modificado = 0, trocas_total_modificado = 0;
        unsigned long soma_duracao_modificado = 0;

        for (int i = 0; i < 10; i++) {
            testar_algoritmo(&args2, elements_copia);
            soma_duracao_modificado += args2.duracao;
            comparacoes_total_modificado += args2.comparacoes;
            trocas_total_modificado += args2.trocas;

            if (args2.duracao < duracao_min_modificado) {
                duracao_min_modificado = args2.duracao;
                args2.min = duracao_min_modificado;
            }
            if (args2.duracao > duracao_max_modificado) {
                duracao_max_modificado = args2.duracao;
                args2.max = duracao_max_modificado;
            }

            memcpy(elements_copia, elements, tamanho * sizeof(int));
        }

        total_duracao_modificado += soma_duracao_modificado / 10;
        total_comparacoes_modificado += comparacoes_total_modificado / 10;
        total_trocas_modificado += trocas_total_modificado / 10;

        free(elements);
        free(elements_copia);
    }

    // Exibindo Resultados
    printf("\n\n\n" NEGrito "RESULTADO DE TESTE DOS ALGORITMOS " MAGENTA ITALIC "s" RESET NEGrito " E " MAGENTA ITALIC "s" RESET NEGrito " COM " GREEN "s" RESET NEGrito " ELEMENTOS\n", nome_func1, nome_func2, tamanho);
    printf("\n\n" CYAN "-----\n" RESET);
    printf("\n" CYAN ITALIC "TAMANHO DO VETOR" RESET " | " CYAN ITALIC "MEDIDA DO TEMPO" RESET " | " CYAN ITALIC "DIFERENÇA MIN E MAX" RESET " | " CYAN ITALIC "MEDIDA DE COMPARACOES" RESET " | " CYAN ITALIC "MEDIDA DE TROCAS" RESET " | \n");
    printf("\n" CYAN "-----\n" RESET);
    printf("\n" CYAN "original" " " GREEN NEGrito "s" RESET " | " GREEN NEGrito "s" RESET " | " GREEN NEGrito "s" RESET " | " GREEN NEGrito "s" RESET " | " GREEN NEGrito "s" RESET " | " GREEN NEGrito "s" RESET " | \n");
    printf("\n" CYAN "tamanho, total_duracao_original / 10, args.max - args.min, total_comparacoes_original / 10, total_trocas_original / 10;\n");
    printf("\n" CYAN "-----\n" RESET);
    printf("\n" CYAN "modificado" " " GREEN NEGrito "s" RESET " | " GREEN NEGrito "s" RESET " | " GREEN NEGrito "s" RESET " | " GREEN NEGrito "s" RESET " | " GREEN NEGrito "s" RESET " | " GREEN NEGrito "s" RESET " | \n");
    printf("\n" CYAN "tamanho, total_duracao_modificado / 10, args2.max - args2.min, total_comparacoes_modificado / 10, total_trocas_modificado / 10;\n");
}
```

- Descrição: A função executar\_teste recebe como parâmetros o tamanho do vetor a ser testado, duas funções de ordenação (func1 e func2), e os nomes dessas funções para identificação nos resultados. Inicialmente, duas estruturas do tipo ArgsIterativo (args e args2) são configuradas para armazenar as informações necessárias para a execução dos testes, incluindo a referência para as funções de ordenação e o tamanho do vetor. A primeira parte do loop realiza 10 testes com a

função de ordenação original (func1). O vetor original é copiado para o vetor `elements_copia`, que será utilizado durante a execução do algoritmo. Em seguida, a função `testar_algoritmo` é chamada para ordenar o vetor, e os resultados são acumulados. Além disso, os valores mínimos e máximos das durações são atualizados conforme necessário. Após cada execução, o vetor copiado é restaurado para seu estado original antes da próxima iteração. Os resultados acumulados são divididos por 10 para calcular as médias ao final do processo.

```
void executar_teste(int tamanho, FuncaoIterativa func1, FuncaoIterativa func2, const char* nome_func1, const char* nome_func2) {
    ArgsIterativa args;
    ArgsIterativa args2;
    args.func = func1;
    args2.func = func2;
    args.n = tamanho;
    args2.n = tamanho;
    args.min = ULONG_MAX; args.max = 0;
    args2.min = ULONG_MAX; args2.max = 0;

    unsigned long total_duracao_original = 0, total_duracao_modificado = 0;
    unsigned long duracao_min_original = ULONG_MAX, duracao_max_original = 0;
    unsigned long total_comparacoes_original = 0, total_comparacoes_modificado = 0;
    unsigned long duracao_min_modificado = ULONG_MAX, duracao_max_modificado = 0;
    unsigned long total_trocas_original = 0, total_trocas_modificado = 0;

    for (int j = 0; j < 10; j++) {
        int *elements = NULL, *elements_copia = NULL;
        alocar_vetor(&elements, tamanho);
        alocar_vetor(&elements_copia, tamanho);

        preencher_vetor(elements, tamanho, 0, tamanho * 10, j);
        memcpy(elements_copia, elements, tamanho * sizeof(int));
        args.vetor = j + 1;
        args2.vetor = j + 1;

        printf(CYAN NEGRITO "\n\n-----\n" RESET);
        printf(CYAN NEGRITO "                        TESTANDO FUNCAO ORIGINAL \n" RESET);
        printf(CYAN NEGRITO "-----" RESET);

        unsigned long comparacoes_total_original = 0, trocas_total_original = 0;
        unsigned long soma_duracao_original = 0;

        for (int i = 0; i < 10; i++) {
            testar_algoritmo(&args, elements_copia);
            soma_duracao_original += args.duracao;
            comparacoes_total_original += args.comparacoes;
            trocas_total_original += args.trocas;

            if (args.duracao < duracao_min_original) {
                duracao_min_original = args.duracao;
                args.min = duracao_min_original;
            }
            if (args.duracao > duracao_max_original) {
                duracao_max_original = args.duracao;
                args.max = duracao_max_original;
            }

            memcpy(elements_copia, elements, tamanho * sizeof(int));
        }

        total_duracao_original += soma_duracao_original / 10;
        total_comparacoes_original += comparacoes_total_original / 10;
        total_trocas_original += trocas_total_original / 10;
    }
}
```

De forma similar ao teste com a função original, o loop é repetido para a função de ordenação modificada (func2). Novamente, dois vetores são alocados, preenchidos e copiados para cada uma das 10 execuções. A função `testar_algoritmo` é chamada para medir o desempenho da função modificada, com os resultados sendo acumulados e as durações mínimas e máximas sendo ajustadas conforme necessário. A cada iteração, o vetor copiado é restaurado, e ao final do loop, as médias de tempo de execução, comparações e trocas são calculadas.

```

printf(CYAN NEGRITO "\n\n-----\n" RESET);
printf(CYAN NEGRITO "                TESTANDO FUNCAO " MAGENTA "MODIFICADA \n" RESET);
printf(CYAN NEGRITO "-----\n" RESET);

unsigned long comparacoes_total_modificado = 0, trocas_total_modificado = 0;
unsigned long soma_duracao_modificado = 0;

for (int i = 0; i < 10; i++) {
    testar_algoritmo(&args2, elements_copia);
    soma_duracao_modificado += args2.duracao;
    comparacoes_total_modificado += args2.comparacoes;
    trocas_total_modificado += args2.trocas;

    if (args2.duracao < duracao_min_modificado) {
        duracao_min_modificado = args2.duracao;
        args2.min = duracao_min_modificado;
    }
    if (args2.duracao > duracao_max_modificado) {
        duracao_max_modificado = args2.duracao;
        args2.max = duracao_max_modificado;
    }
    memcpy(elements_copia, elements, tamanho * sizeof(int));
}

total_duracao_modificado += soma_duracao_modificado / 10;
total_comparacoes_modificado += comparacoes_total_modificado / 10;
total_trocas_modificado += trocas_total_modificado / 10;

free(elements);
free(elements_copia);

```

Após a execução de todos os testes, os resultados finais são exibidos em um formato tabular. As informações são apresentadas de maneira clara, utilizando formatação e cores ANSI para destacar os diferentes aspectos dos resultados. Esse relatório final permite uma análise direta e comparativa da eficiência entre a função de ordenação original e a função modificada, oferecendo uma visão detalhada do desempenho de ambos os algoritmos.

```

printf("\n\n" NEGRITO "RESULTADO DE TESTE DOS ALGORITMOS " MAGENTA ITALIC "%s" RESET NEGRITO " E " MAGENTA ITALIC "%s" RESET NEGRITO " COM " GREEN "%d" RESET NEGRITO " ELEMENTOS\n", nome_func1, nome_func2, tamanho);
printf("\n" CYAN "-----" RESET);
printf("\n" CYAN ITALIC "ALGORITMO" RESET " | " CYAN ITALIC "TAMANHO DO VETOR" RESET " | " CYAN ITALIC "MEDIA DO TEMPO" RESET " | " CYAN ITALIC "DIFERENCA MIN E MAX" RESET " | " CYAN ITALIC "MEDIA DE COMPARACOES" RESET " | " CYAN ITALIC "MEDIA DE TROCAS" RESET " | \n");
printf("\n" CYAN "-----" RESET);
printf("\n" CYAN "original | " GREEN NEGRITO "%d" RESET " | " GREEN NEGRITO "%10s" RESET " | " GREEN NEGRITO "%10s" RESET " | " GREEN NEGRITO "%10s" RESET " | " GREEN NEGRITO "%10s" RESET " | " GREEN NEGRITO "%10s" RESET " | \n",
    tamanho, total_duracao_original / 10, args.max - args.min, total_comparacoes_original / 10, total_trocas_original / 10);
printf("\n" CYAN "-----" RESET);
printf("\n" CYAN "modificada | " GREEN NEGRITO "%d" RESET " | " GREEN NEGRITO "%10s" RESET " | " GREEN NEGRITO "%10s" RESET " | " GREEN NEGRITO "%10s" RESET " | " GREEN NEGRITO "%10s" RESET " | " GREEN NEGRITO "%10s" RESET " | \n",
    tamanho, total_duracao_modificado / 10, args2.max - args2.min, total_comparacoes_modificado / 10, total_trocas_modificado / 10);

```

- Entrada:
  - int tamanho: Tamanho dos vetores a serem usados nos testes.
  - void FuncaoIterativa func1: Ponteiro de uma função para o primeiro algoritmo de ordenação.

- void FuncaoIterativa func2: Ponteiro de uma função para o segundo algoritmo de ordenação.
- const char\* nome\_func1: Nome do primeiro algoritmo (para exibição).
- const char\* nome\_func2: Nome do segundo algoritmo (para exibição).
- Saída: Nenhuma (a função imprime os resultados no console).

## 4.2 Biblioteca.h

Este arquivo de cabeçalho define estruturas e funções essenciais para testar e comparar algoritmos de ordenação iterativos e recursivos. Ele permite a execução dos testes em threads separadas, medições detalhadas de desempenho, e facilita a visualização dos resultados. As estruturas ArgsIteratctive e ArgsRecursive encapsulam os dados necessários para cada tipo de algoritmo, e funções como alocar\_vetor, percorrer\_vetor, e imprimir\_vetor fornecem utilitários básicos para manipulação dos vetores.

```
#ifndef MEU_ARQUIVO_H
#define MEU_ARQUIVO_H
#include <windows.h>
//===== STRUCTURES =====
typedef void (*FuncaoIterativa)(int*, int, unsigned long*, unsigned long*);
typedef void (*FuncaoRecursiva)(int*, int,int, unsigned long*, unsigned long*);

typedef struct {
    FuncaoIterativa func;
    int n;
    unsigned long int duracao;
    unsigned long int comparacoes;
    unsigned long int trocas;
    int max;
    int min;
    int vetor;
} ArgsIteratctive;

typedef struct{
    FuncaoRecursiva func;
    int n;
    unsigned long int duracao;
    unsigned long int comparacoes;
    unsigned long int trocas;
    int max;
    int min;
    int vetor;
} ArgsRecursive;

void alocar_vetor(int **vetor, int n);
void percorrer_vetor(int *vetor,int n);
void imprimir_vetor(const int *vetor, int n);
void testar_algoritmo(ArgsIteratctive* args, int * elements);
void executar_teste(int tamanho, FuncaoIterativa func1, FuncaoIterativa func2, const char* nome_func1, const char* nome_func2);

#endif
```

#### **4.2.1. typedef void (\*FuncaoIterativa)(int\*, int, unsigned long\*, unsigned long\*):**

Define um ponteiro para funções de ordenação iterativas, que recebem um vetor de inteiros, seu tamanho, e dois ponteiros para contagem de comparações e trocas.

#### **4.2.2 typedef void (\*FuncaoRecursiva)(int\*, int, int, unsigned long\*, unsigned long\*):**

Define um ponteiro para funções de ordenação recursivas, que além dos parâmetros anteriores, recebem um parâmetro adicional para o índice inicial ou limite da recursão (OBS: esse ponteiro foi somente usado para argumentos das funções quickSort e margeSort, porém foram descontinuadas).

**4.2.3 void alocar\_vetor(int \*\*vetor, int n):** Aloca memória dinamicamente para um vetor de inteiros de tamanho n.

**4.2.4 void preencher\_vetor(int \*vetor, int n):** executa a inserção de n elementos inteiros aleatórios no vetor;

**4.2.5 void imprimir\_vetor(const int \*vetor, int n):** Imprime os elementos de um vetor de inteiros de tamanho n.

**4.2.6 void testar\_algoritmo(Argslteratctive\* args, int \* elements):**Executa um algoritmo de ordenação iterativo utilizando os parâmetros da estrutura Argslteratctive, medindo o tempo de execução, o número de comparações e trocas realizadas, e atualiza esses valores na estrutura.

**4.2.7 void executar\_teste(int tamanho, FuncaoIterativa func1, FuncaoIterativa func2, const char\* nome\_func1, const char\* nome\_func2):** Executa testes comparativos entre duas funções de ordenação iterativas, func1 e func2, para um vetor de tamanho tamanho. A função realiza múltiplas execuções, coleta e calcula médias de desempenho, e gera um relatório final comparando os resultados das duas funções, destacando aspectos como tempo de execução e eficiência.



## 4.3 Main.c

```
// # 1º Escolher dois algoritmos de ordenação entre os algoritmos abaixo
//      (BubbleSort, MergeSort, InsertSort, insertionSort, QuickSort)
//
//      Após isso, perguntar a algum modelo de LLM (CHATGPT LLHAMA, COPILOT, BING)
//      como tornar tal algoritmo mais eficiente, em seguida testá-los e fazer uma
//      análise crítica sobre tais alterações e escolher uma variação para cada código
//      JUSTIFICANDO A SUA ESCOLHA!!!
//
// # 2º Para cada algoritmo, compare a sua complexidade por análise de comportamento
//      assintótico usando a notação Big(O).
//      Depois faz uma análise empírica, implementando um gerador de inteiros aleatórios
//      podendo retornar um vetor de tamanho N (10,100,1.000,10.000)

#include "biblioteca.c"
#include "ordination.h"
#include "ordination.c"

int main() {
    boxLoadingAnimation(80);
    executar_teste(10, bubblesort, bubblesortM, "bubble", "buibble");
    executar_teste(100, bubblesort, bubblesortM, "bubble", "buibble");
    executar_teste(1000, bubblesort, bubblesortM, "bubble", "buibble");
    executar_teste(10000, bubblesort, bubblesortM, "bubble", "buibble");

    return 0;
}
```

Este arquivo será o main principal de execução do projeto de teste de algoritmos. Estes comandos incluem os arquivos necessários para o funcionamento do programa. biblioteca.c provavelmente contém funções auxiliares ou utilitárias, enquanto ordination.h é o arquivo de cabeçalho que declara as funções e tipos relacionados à ordenação. ordination.c contém as implementações dessas funções.

---

## 4.4 ordination.c

Nesse arquivo estão presentes os algoritmos de ordenação, tanto os originais quanto os modificados.

#### 4.4.1 Função void swap(int \*x, int \*y)

```
void swap(int *x, int *y){  
    int temp = *x;  
    *x = *y;  
    *y = temp;  
}
```

- **Descrição:** A função swap troca os valores de dois elementos inteiros. Essa função é amplamente utilizada em algoritmos de ordenação para permutar elementos que não estão na ordem correta.
  - **Entrada:**
    - int \*x: Ponteiro para o primeiro elemento.
    - int \*y: Ponteiro para o segundo elemento.
  - **Saída:** Nenhuma (a função realiza a troca dos valores entre os dois elementos apontados por x e y).
- 

#### 4.4.2 Função void bubblesort(int \*vet, int n, unsigned long \*comparacoes, unsigned long \*trocas)

```
void bubblesort(int *vet, int n, unsigned long *comparacoes, unsigned long *trocas){  
    int i, mudou, fim = n-1;  
    *trocas = 0;  
    *comparacoes = 0;  
    do{  
        mudou = 0;  
        for(i = 0; i < fim; i++){  
            (*comparacoes)++;  
            if(vet[i] > vet[i+1]){  
                swap(&vet[i], &vet[i+1]);  
                mudou = 1;  
                (*trocas)++;  
            }  
        }  
        fim--;  
    }while(mudou);  
}
```

- **Descrição:** A função bubblesort implementa o algoritmo de ordenação Bubble Sort, que compara pares de elementos adjacentes e os troca se estiverem na ordem errada. O processo é repetido até que o vetor esteja ordenado. Durante a execução, a função também conta o número de comparações e trocas realizadas.
  - **Entrada:**
    - int \*vet: Ponteiro para o vetor que será ordenado.
    - int n: Tamanho do vetor.
    - unsigned long \*comparacoes: Ponteiro para a variável que armazena o número de comparações realizadas.
    - unsigned long \*trocas: Ponteiro para a variável que armazena o número de trocas realizadas.
  - **Saída:** Nenhuma (a função ordena o vetor in-place e atualiza as contagens de comparações e trocas).
- 

#### 4.4.3 Função void insertionSort(int \*vet, int n, unsigned long \*comparacoes, unsigned long \*trocas)

```
void insertionSort(int *vet, int n, unsigned long *comparacoes, unsigned long *trocas){
    int i, j, aux;
    *trocas = 0;
    *comparacoes = 0;
    for(i = 1; i < n; i++){
        aux = vet[i];
        j = i;
        while(j > 0 && aux < vet[j-1]){
            (*comparacoes)++;
            vet[j] = vet[j-1];
            j--;
            (*trocas)++;
        }
        vet[j] = aux;
        if (j != i) {
            (*trocas)++;
        }
    }
}
```

- **Descrição:** A função insertionSort implementa o algoritmo de ordenação Insertion Sort, que constrói a ordenação um elemento por vez, inserindo cada novo elemento na posição correta em relação aos elementos já ordenados. Durante a execução, a função também conta o número de comparações e trocas realizadas.
  - **Entrada:**
    - int \*vet: Ponteiro para o vetor que será ordenado.
    - int n: Tamanho do vetor.
    - unsigned long \*comparacoes: Ponteiro para a variável que armazena o número de comparações realizadas.
    - unsigned long \*trocas: Ponteiro para a variável que armazena o número de trocas realizadas.
  - **Saída:** Nenhuma (a função ordena o vetor in-place e atualiza as contagens de comparações e trocas).
- 

#### 4.4.4 Função void bubblesortM(int \*vet, int n, unsigned long \*comparacoes, unsigned long \*trocas)

```
void bubblesortM(int *vet, int n, unsigned long *comparacoes, unsigned long *trocas) {
    int i, mudou;
    *trocas = 0;
    *comparacoes = 0;
    int sorted = 0;
    while (!sorted) {
        sorted = 1;
        for (i = 0; i <= n - 2; i += 2) {
            (*comparacoes)++;
            if (vet[i] > vet[i + 1]) {
                swap(&vet[i], &vet[i + 1]);
                sorted = 0;
                (*trocas)++;
            }
        }
        for (i = 1; i <= n - 2; i += 2) {
            (*comparacoes)++;
            if (vet[i] > vet[i + 1]) {
                swap(&vet[i], &vet[i + 1]);
                sorted = 0;
                (*trocas)++;
            }
        }
    }
}
```

- **Descrição:** A função bubblesortM é uma variação modificada do algoritmo Bubble Sort, onde a ordenação é feita em duas fases. A primeira fase compara e troca elementos nos índices pares, e a segunda fase faz o mesmo para os índices ímpares. O processo é repetido até que o vetor esteja ordenado. Durante a execução, a função conta o número de comparações e trocas realizadas.
- **Entrada:**
  - int \*vet: Ponteiro para o vetor que será ordenado.
  - int n: Tamanho do vetor.
  - unsigned long \*comparacoes: Ponteiro para a variável que armazena o número de comparações realizadas.
  - unsigned long \*trocas: Ponteiro para a variável que armazena o número de trocas realizadas.
- **Saída:** Nenhuma (a função ordena o vetor in-place e atualiza as contagens de comparações e trocas).

#### 4.4.5 Função void insertionSortM(int \*vet, int n, unsigned long \*comparacoes, unsigned long \*trocas)

```
void insertionSortM(int *vet, int n, unsigned long *comparacoes, unsigned long *trocas) {
    int i, j, aux, pos;
    *trocas = 0;
    *comparacoes = 0;
    for (i = 1; i < n; i++) {
        aux = vet[i];
        int low = 0, high = i;
        while (low < high) {
            int mid = (low + high) / 2;
            (*comparacoes)++;
            if (vet[mid] <= aux) {
                low = mid + 1;
            } else {
                high = mid;
            }
        }
        pos = low;
        for (j = i; j > pos; j--) {
            vet[j] = vet[j - 1];
            (*trocas)++;
        }
        vet[pos] = aux;
        if (pos != i) {
            (*trocas)++;
        }
    }
}
```

- **Descrição:** A função `insertionSortM` é uma versão otimizada do algoritmo Insertion Sort, que utiliza uma busca binária para encontrar a posição correta de inserção do elemento. Isso reduz o número de comparações necessárias. Após encontrar a posição, os elementos são deslocados para abrir espaço para o novo elemento. Durante a execução, a função conta o número de comparações e trocas realizadas.
- **Entrada:**
  - `int *vet`: Ponteiro para o vetor que será ordenado.
  - `int n`: Tamanho do vetor.
  - `unsigned long *comparacoes`: Ponteiro para a variável que armazena o número de comparações realizadas.
  - `unsigned long *trocas`: Ponteiro para a variável que armazena o número de trocas realizadas.
- **Saída:** Nenhuma (a função ordena o vetor in-place e atualiza as contagens de comparações e trocas).

## 4.5 ordination.h

Este arquivo de cabeçalho `ordenacao.h` declara as funções para diferentes algoritmos de ordenação, incluindo versões básicas e modificadas dos algoritmos Bubble Sort e Insertion Sort. Ele também declara uma função auxiliar `swap` para troca de elementos. Esse arquivo é utilizado para garantir que essas funções possam ser chamadas a partir de outros arquivos no projeto.

```
#ifndef ORDENACAO_H
#define ORDENACAO_H

void swap(int *x, int *y);
void bubblesort(int *vet, int n, unsigned long *trocas, unsigned long *comparacoes);
void insertionSort(int *vetor, int n, unsigned long *comparacoes, unsigned long *trocas);

//+++++MODIFICADOS+++++
void bubblesortM(int *vet, int n, unsigned long *trocas, unsigned long *comparacoes);
void insertionSortM(int *vet, int n, unsigned long *comparacoes, unsigned long *trocas);

#endif
```

---

#### 4.5.1 Função void swap (int \*x, int \*y)

- **Descrição:** Troca os valores de dois elementos inteiros, sendo amplamente utilizada em algoritmos de ordenação.

---

#### 4.5.2 Função void bubblesort(int \*vet, int n, unsigned long \*trocas, unsigned long \*comparacoes)

- **Descrição:** Implementa o algoritmo de ordenação Bubble Sort, que compara e troca elementos adjacentes até que o vetor esteja ordenado. Também contabiliza o número de trocas e comparações realizadas.

---

#### 4.5.3 Função void insertionSort(int \*vetor, int n, unsigned long \*comparacoes, unsigned long \*trocas)

- **Descrição:** Implementa o algoritmo de ordenação Insertion Sort, que insere cada elemento na posição correta, mantendo a ordenação parcial. Contabiliza o número de comparações e trocas.

---

#### 4.5.4 Função void bubblesortM(int \*vet, int n, unsigned long \*trocas, unsigned long \*comparacoes)

- **Descrição:** Uma versão modificada do Bubble Sort, onde a ordenação é feita em duas fases: a primeira para elementos em índices pares e a segunda para elementos em índices ímpares. Contabiliza trocas e comparações.

---

#### 4.5.5 Função void insertionSortM(int \*vet, int n, unsigned long \*comparacoes, unsigned long \*trocas)

- **Descrição:** Uma versão otimizada do Insertion Sort, que utiliza busca binária para determinar a posição correta de inserção do elemento, reduzindo comparações. Também contabiliza o número de trocas.

## 4.6 style.c

Este arquivo possui apenas o código da animação de carregamento, tendo como cabeçalho algumas contantes de estilização.

[illegible]

#### 4.6.1 Função void boxLoadingAnimation(int duration)

- **Descrição:** Esta função implementa uma animação de carregamento em um estilo de "barra de progresso" que é exibida no terminal. A barra de progresso avança e



retrocede em ciclos até que o tempo de duração especificado seja alcançado. A animação utiliza cores (verde) para destacar a barra de progresso e é formatada para ser exibida em tempo real no console.

## 4.7 Style.h

Este arquivo define uma série de macros que representam códigos de cores e estilos de texto ANSI, que podem ser usados para estilizar a saída do terminal. Além disso, o arquivo também declara a função `boxLoadingAnimation`, que é utilizada para exibir uma animação de carregamento no terminal.

```
#ifndef STYLE_H
#define STYLE_H

#define BLUE          "\x1b[34m"
#define MAGENTA       "\x1b[35m"
#define CYAN          "\x1b[36m"
#define RESET         "\x1b[0m"
#define NEGRITO       "\x1b[1m"
#define DIMINUIDO     "\x1b[2m"
#define SUBLINHADO     "\x1b[4m"
#define ITALIC        "\x1b[3m"
#define MAGENTA_BG    "\x1b[45m"
#define CYAN_BG       "\x1b[46m"
#define WHITE_BG      "\x1b[47m"
#define BLACK         "\x1b[30m"
#define RED           "\x1b[31m"
#define YELLOW        "\x1b[33m"
#define WHITE         "\x1b[37m"
#define BRIGHT_BLACK  "\x1b[90m"
#define BRIGHT_RED    "\x1b[91m"
#define BRIGHT_GREEN  "\x1b[92m"
#define BRIGHT_YELLOW "\x1b[93m"
#define BRIGHT_BLUE   "\x1b[94m"
#define BRIGHT_CYAN   "\x1b[96m"
#define BRIGHT_WHITE  "\x1b[97m"
#define BRIGHT_BLACK_BG "\x1b[100m"
#define BRIGHT_RED_BG  "\x1b[101m"
#define BRIGHT_GREEN_BG "\x1b[102m"
#define BRIGHT_YELLOW_BG "\x1b[103m"
#define BRIGHT_BLUE_BG "\x1b[104m"
#define BRIGHT_MAGENTA_BG "\x1b[105m"
#define BRIGHT_CYAN_BG "\x1b[106m"
#define BRIGHT_WHITE_BG "\x1b[107m"
#define GREEN         "\x1b[32m"
```

As macros definidas representam sequências de escape ANSI, que permitem controlar a cor do texto e o fundo, além de aplicar estilos como negrito e sublinhado na saída do terminal. O uso dessas macros facilita a aplicação de estilos em programas que imprimem mensagens no console, tornando o código mais legível e reutilizável.

#### 4.7.1 Função void boxLoadingAnimation(int duration)

```
void boxLoadingAnimation(int duration);  
#endif
```

- **Descrição:** Declara a função boxLoadingAnimation, que exibe uma animação de carregamento no terminal.
- **Entrada:**
  - int duration: Duração da animação, medida em unidades de tempo (cada unidade é um ciclo da animação).
- **Saída:** Nenhuma (a função imprime a animação diretamente no console).

### Execução do teste

No teste, será rodado o algoritmo 4.1.4 para cada quantidade de valores do vetor para cada tipo de algoritmo de ordenação separadamente, terminando semelhante a uma tabela visível pelo terminal.

Cada um dos 10 vetores aleatórios serão ordenados pelos algoritmos originais e modificados d10 vezes como explicado na descrição do algoritmo 4.1.5, resultando num output semelhante ao mostrado abaixo:

```

-----
TESTANDO FUNCAO ORIGINAL
-----
EXECUTANDO | vetor 10 | 174 ms | 189 - 172 = 17 | 49988672 comparacoes | 25227847 trocas.
=====
EXECUTANDO | vetor 10 | 174 ms | 189 - 172 = 17 | 49988672 comparacoes | 25227847 trocas.
=====
EXECUTANDO | vetor 10 | 174 ms | 189 - 172 = 17 | 49988672 comparacoes | 25227847 trocas.
=====
EXECUTANDO | vetor 10 | 174 ms | 189 - 172 = 17 | 49988672 comparacoes | 25227847 trocas.
=====
EXECUTANDO | vetor 10 | 176 ms | 189 - 172 = 17 | 49988672 comparacoes | 25227847 trocas.
=====
EXECUTANDO | vetor 10 | 179 ms | 189 - 172 = 17 | 49988672 comparacoes | 25227847 trocas.
=====
EXECUTANDO | vetor 10 | 178 ms | 189 - 172 = 17 | 49988672 comparacoes | 25227847 trocas.
=====
EXECUTANDO | vetor 10 | 177 ms | 189 - 172 = 17 | 49988672 comparacoes | 25227847 trocas.
=====
EXECUTANDO | vetor 10 | 173 ms | 189 - 172 = 17 | 49988672 comparacoes | 25227847 trocas.
=====
EXECUTANDO | vetor 10 | 176 ms | 189 - 172 = 17 | 49988672 comparacoes | 25227847 trocas.
=====

-----
TESTANDO FUNCAO MODIFICADA
-----

EXECUTANDO | vetor 10 | 158 ms | 165 - 152 = 13 | 49635036 comparacoes | 25227847 trocas.
=====
EXECUTANDO | vetor 10 | 156 ms | 165 - 152 = 13 | 49635036 comparacoes | 25227847 trocas.
=====
EXECUTANDO | vetor 10 | 159 ms | 165 - 152 = 13 | 49635036 comparacoes | 25227847 trocas.
=====
EXECUTANDO | vetor 10 | 157 ms | 165 - 152 = 13 | 49635036 comparacoes | 25227847 trocas.
=====
EXECUTANDO | vetor 10 | 159 ms | 165 - 152 = 13 | 49635036 comparacoes | 25227847 trocas.
=====
EXECUTANDO | vetor 10 | 156 ms | 165 - 152 = 13 | 49635036 comparacoes | 25227847 trocas.
=====
EXECUTANDO | vetor 10 | 162 ms | 165 - 152 = 13 | 49635036 comparacoes | 25227847 trocas.
=====
EXECUTANDO | vetor 10 | 155 ms | 165 - 152 = 13 | 49635036 comparacoes | 25227847 trocas.
=====
EXECUTANDO | vetor 10 | 158 ms | 165 - 152 = 13 | 49635036 comparacoes | 25227847 trocas.
=====
EXECUTANDO | vetor 10 | 162 ms | 165 - 152 = 13 | 49635036 comparacoes | 25227847 trocas.
=====

RESULTADO DE TESTE DOS ALGORITMOS bubble E buibble COM 10000 ELEMENTOS
-----
| ALGORITMO | TAMANHO DO VETOR | MEDIA DO TEMPO | DIFERENÇA MIN E MAX | MEDIA DE COMPARACOES | MEDIA DE TROCAS |
|-----|-----|-----|-----|-----|-----|
| original | 10000 | 176 | 17 | 49986413 | 25020904 |
|-----|-----|-----|-----|-----|-----|
-|
| modificada | 10000 | 157 | 13 | 49495050 | 25020904 |
|-----|-----|-----|-----|-----|-----|

```

Para executar o teste, é compilado o código para um executável usando o comando “**gcc ordination.h biblioteca.h style.h main.c -o main.exe**” seguido de um comando “**./main.exe**” pelo Vscode. Você pode usar outras alternativas como o CodeBlocks.

## 5. Discursão dos resultados

Sobre os resultados dos algoritmos de forma empírica temos os seguintes resultados:

### BubbleSort – 10 elementos:

| RESULTADO DE TESTE DOS ALGORITMOS <i>bubble</i> E <i>buibble</i> COM 10 ELEMENTOS |                  |                |                     |                      |                 |  |
|-----------------------------------------------------------------------------------|------------------|----------------|---------------------|----------------------|-----------------|--|
| ALGORITMO                                                                         | TAMANHO DO VETOR | MEDIA DO TEMPO | DIFERENÇA MIN E MAX | MEDIA DE COMPARACOES | MEDIA DE TROCAS |  |
| original                                                                          | 10               | 0              | 0                   | 41                   | 25              |  |
| modificada                                                                        | 10               | 0              | 0                   | 47                   | 25              |  |

O código é executado tão rápido nos dois algoritmos que é menor do que milissegundos, sendo o tempo mínimo e máximo também nulos. Nota-se que o algoritmo original possui menos comparações em média em relação ao código modificado, mesmo com a mesma média de trocas por vetor.

### BubbleSort – 100 elementos:

| RESULTADO DE TESTE DOS ALGORITMOS <i>bubble</i> E <i>buibble</i> COM 100 ELEMENTOS |                  |                |                     |                      |                 |  |
|------------------------------------------------------------------------------------|------------------|----------------|---------------------|----------------------|-----------------|--|
| ALGORITMO                                                                          | TAMANHO DO VETOR | MEDIA DO TEMPO | DIFERENÇA MIN E MAX | MEDIA DE COMPARACOES | MEDIA DE TROCAS |  |
| original                                                                           | 100              | 0              | 1                   | 4887                 | 2456            |  |
| modificada                                                                         | 100              | 0              | 1                   | 4633                 | 2456            |  |

O código é executado tão rápido nos dois algoritmos que é menor do que milissegundos, porém tendo algumas execuções que chegam aos 1 milissegundos, sendo o tempo mínimo e máximo também nulos. Nota-se que o algoritmo original começou a ter a média de comparações maior em relação ao modificado com o aumento da quantidade de valores do vetor, mesmo que a média de trocas seja a mesma.

## BubbleSort – 1.000 elementos:

RESULTADO DE TESTE DOS ALGORITMOS *bubble* E *bubble* COM 1000 ELEMENTOS

| ALGORITMO  | TAMANHO DO VETOR | MEDIA DO TEMPO | DIFERENÇA MIN E MAX | MEDIA DE COMPARACOES | MEDIA DE TROCAS |
|------------|------------------|----------------|---------------------|----------------------|-----------------|
| original   | 1000             | 1              | 2                   | 499132               | 248047          |
| modificada | 1000             | 1              | 2                   | 491008               | 248047          |

O código é executado tão rápido nos dois algoritmos que é executado entre 1 á 2 milissegundos, tendo rara variações que levam em 3milissegundos no mesmo vetor que levar 2milissegundos, dependendo da sua execução (isso devido ao algoritmo ordenar o mesmo vetor 10 vezes). Como a média de tempo não varia muito, a diferença entre o tempo máximo e mínimo também possui o mesmo comportamento. A média das trocas continua igual para ambos os algoritmos, e a média de comparações continua sendo maior para o original.

## BubbleSort – 10.000 elementos:

RESULTADO DE TESTE DOS ALGORITMOS *bubble* E *bubble* COM 10000 ELEMENTOS

| ALGORITMO  | TAMANHO DO VETOR | MEDIA DO TEMPO | DIFERENÇA MIN E MAX | MEDIA DE COMPARACOES | MEDIA DE TROCAS |
|------------|------------------|----------------|---------------------|----------------------|-----------------|
| original   | 10000            | 183            | 61                  | 49989502             | 25010950        |
| modificada | 10000            | 172            | 64                  | 49636035             | 25010950        |

PS: C:\Users\carbas\Documents\GitHub\CLASD\05 - primeiroProjeto - main.py

RESULTADO DE TESTE DOS ALGORITMOS *bubble* E *bubble* COM 10000 ELEMENTOS

| ALGORITMO  | TAMANHO DO VETOR | MEDIA DO TEMPO | DIFERENÇA MIN E MAX | MEDIA DE COMPARACOES | MEDIA DE TROCAS |
|------------|------------------|----------------|---------------------|----------------------|-----------------|
| original   | 10000            | 183            | 86                  | 49982243             | 24961294        |
| modificada | 10000            | 170            | 14                  | 49498049             | 24961294        |

Nessa quantidade de elementos no vetor, algumas coisas são importantes ressaltar. O tempo de execução é menor no algoritmo modificado, junto com a média de comparações, entretanto a média de trocas continua a mesma e a diferença entre o tempo máximo e mínimo varia bastante, mesmo com o código testando o algoritmo 10 vezes para cada um dos 10 vetores. Como mostrado nas imagens, a função modificada tem uma diferença maior, porem na segunda execução do teste a função original é leva mais tempo em relação ao modificado.

## InsertionSort – 10 elementos:

| RESULTADO DE TESTE DOS ALGORITMOS <i>insertion</i> E <i>insertion modificaded</i> COM 10 ELEMENTOS |                  |                |                     |                      |                 |  |
|----------------------------------------------------------------------------------------------------|------------------|----------------|---------------------|----------------------|-----------------|--|
| ALGORITMO                                                                                          | TAMANHO DO VETOR | MEDIA DO TEMPO | DIFERENÇA MIN E MAX | MEDIA DE COMPARACOES | MEDIA DE TROCAS |  |
| original                                                                                           | 10               | 0              | 0                   | 17                   | 23              |  |
| modificada                                                                                         | 10               | 0              | 0                   | 21                   | 23              |  |

O código é executado tão rápido nos dois algoritmos que é menor do que milissegundos, sendo o tempo mínimo e máximo também nulos. Nota-se que o algoritmo original possui menos comparações em média em relação ao código modificado, mesmo com a mesma média de trocas por vetor.

## InsertionSort – 100 elementos:

| RESULTADO DE TESTE DOS ALGORITMOS <i>insertion</i> E <i>insertion modificaded</i> COM 100 ELEMENTOS |                  |                |                     |                      |                 |  |
|-----------------------------------------------------------------------------------------------------|------------------|----------------|---------------------|----------------------|-----------------|--|
| ALGORITMO                                                                                           | TAMANHO DO VETOR | MEDIA DO TEMPO | DIFERENÇA MIN E MAX | MEDIA DE COMPARACOES | MEDIA DE TROCAS |  |
| original                                                                                            | 100              | 0              | 1                   | 2413                 | 2507            |  |
| modificada                                                                                          | 100              | 0              | 1                   | 530                  | 2507            |  |

O código é executado tão rápido nos dois algoritmos que é menor do que milissegundos, porém tendo algumas execuções que chegam aos 1 milissegundos, sendo o tempo mínimo e máximo no máximo até 1 milissegundo. Nota-se que o algoritmo modificado começou a ter a média de comparações menor em relação ao modificado com o aumento da quantidade de valores do vetor, mesmo que a média de trocas seja a mesma.

## InsertionSort – 1.000 elementos:

| RESULTADO DE TESTE DOS ALGORITMOS <i>insertion</i> E <i>insertion modificaded</i> COM 1000 ELEMENTOS |                  |                |                     |                      |                 |  |
|------------------------------------------------------------------------------------------------------|------------------|----------------|---------------------|----------------------|-----------------|--|
| ALGORITMO                                                                                            | TAMANHO DO VETOR | MEDIA DO TEMPO | DIFERENÇA MIN E MAX | MEDIA DE COMPARACOES | MEDIA DE TROCAS |  |
| original                                                                                             | 1000             | 0              | 2                   | 250214               | 251206          |  |
| modificada                                                                                           | 1000             | 0              | 2                   | 8584                 | 251206          |  |

O código é executado tão rápido nos dois algoritmos que é executado entre 0 a 1 milissegundos, raramente chegando aos 2 milissegundos. Como a média de tempo não varia muito, a diferença entre o tempo máximo e mínimo também possui o mesmo

comportamento. A média das trocas continua igual para ambos os algoritmos, e a média de comparações continua sendo maior para o original, porem em uma porcentagem muito significativa, diferente do bubbleSort, onde tinha diferença, porem equilibrada.

## InsertionSort – 10.000 elementos:

| RESULTADO DE TESTE DOS ALGORITMOS <i>insertion</i> E <i>insertion modificada</i> COM 10000 ELEMENTOS |                  |                |                     |                      |                 |  |
|------------------------------------------------------------------------------------------------------|------------------|----------------|---------------------|----------------------|-----------------|--|
| ALGORITMO                                                                                            | TAMANHO DO VETOR | MEDIA DO TEMPO | DIFERENÇA MIN E MAX | MEDIA DE COMPARACOES | MEDIA DE TROCAS |  |
| original                                                                                             | 10000            | 67             | 14                  | 25015318             | 25025307        |  |
| modificada                                                                                           | 10000            | 38             | 27                  | 118988               | 25025307        |  |

Nessa quantidade de elementos no vetor, algumas coisas são importantes ressaltar. O tempo de execução é menor no algoritmo modificado, junto com a média de comparações, entretanto a média de trocas continua a mesma e a diferença entre o tempo máximo e mínimo é menor no algoritmo original.

Sabendo desses resultados, podemos concluir que através da análise empírica, os algoritmos modificados possuem um desempenho igual ou pior para entradas de pequenas quantidades em questão de comparações, porem com o aumento dos elementos de entrada, o tempo de execução é maior nos códigos clássicos junto com o número de comparações.

Comparando os resultados das duas variações, a versão alternativa do insertionSort teve mais desempenho em relação ao original do que o bubbleSort, sendo visível a otimização da média de comparações e do tempo de execução consequentemente. No bubbleSort, a diferença não é tão impactante quanto a do insertinoSort, porém demonstra ser mais eficiente do que o algoritmo original.

Sobre os resultados em análise matemática, no pior caso do BubbleSort modificado, o vetor está em ordem inversa (completamente desordenado). A cada iteração, o algoritmo precisa fazer  $n/2$  comparações para os elementos de índices pares e  $n/2$  comparações para os elementos de índices ímpares, totalizando  $n-1$  comparações por iteração. Em cada iteração, o algoritmo faz aproximadamente  $n$  comparações (somando as comparações dos pares e ímpares).

No pior caso, o número total de comparações seria da ordem de  $n * (n-1) / 2$ , o que resulta em uma complexidade de tempo de  **$O(n^2)$** . Cada iteração pode resultar em até  $n/2$  trocas. Assim, no pior caso, a quantidade de trocas também é da ordem de  $n * (n-1) / 2$ . Dessa forma, a complexidade  **$O(n^2)$** .

No caso do InsertionSort Modificado, fica mais clara o desempenho em análise matemática pela substituição de busca normal por uma busca binária. Para cada iteração do loop principal, o algoritmo executa uma busca binária para encontrar a posição pos onde o elemento vetor[i] deve ser inserido. A busca binária tem uma complexidade de tempo de  $O(\log(n))$ , onde n é o índice atual no loop principal. Para cada iteração do loop principal, o número de comparações feitas pela busca binária é  $O(n\log(n))$  portanto, o número total de comparações no pior caso será a soma das comparações em todas as iterações:

$$\sum n\log(n)$$

Essa soma é aproximadamente igual a  $O(n\log(n))$  no pior caso. A principal vantagem sobre a versão clássica é a redução no número de comparações devido ao uso da busca binária, o que melhora a complexidade de tempo para comparações, mas a complexidade total ainda é dominada pelo deslocamento dos elementos, que permanece  $O(n^2)$ .

## 6 Conclusão

Nesse tópico serão respondidas algumas perguntas em questão ao objetivo dessa pesquisa.

### **Os algoritmos modificados são realmente mais eficientes do que os originais?**

Sim, são mais eficientes vindo no contexto de uma análise empírica, ou seja, pelo tempo de execução, por mais que pertençam à mesma complexidade do algoritmo original pelo ponto de vista da notação BigO, por ambos os códigos serem de complexidade  $O(n^2)$  no pior caso.



## **Por que foram escolhidos o BubbleSort e o InsertionSort para a procura de uma maior performance?**

Diferente do QuickSort, HeapSort e MergeSort, esses dois algoritmos por mais que sejam bastante conhecidos e de fácil compreensão, não são muito eficientes em ordenação de escalas maiores. Eles são uma ótima escolha para um aprimoramento de performance por serem os algoritmos menos performáticos em comparação aos outros.

## **O ChatGPT é recomendável para melhorar eficiência de código?**

Depende. Como um modelo de LLM precisa de, além de uma enorme quantidade de dados, tempo para o treinamento do modelo. As capacidades do ChatGPT podem crescer dependendo da forma que ele é treinado. Entretanto, vale ressaltar que como uma ferramenta para um propósito de resultado imediato não é recomendado o seu uso dependendo do problema apresentado à IA, pois diversas vezes quando era perguntado alguma variante dos algoritmos de ordenação, maioria não tinha uma eficiência significativa, e em alguns casos piorando a eficiência em comparação ao original.

É recomendável o uso da ferramenta para estudo de novas tecnologias e problemas simples, mas não para atividades complexas.

## **Os algoritmos modificados ainda podem se tornar mais eficientes?**

No caso dos algoritmos escolhidos, possivelmente sim, porem precisará muito mais do que um modelo de LLM para procurar tal performance, já no caso dos outros algoritmos de ordenação variados dos códigos clássicos, depende. Assim como a computação no geral, diversos modelos são aprimorados ou inventados com o passar dos anos. Entretanto existem certos modelos no qual possuem um limite de aprimoramento, por mais que a evolução contante dos algoritmos demonstra sempre um aprimoramento em fundamentos antigos.